Adaptive Reasoning and Test-Time Computation: A Multi-Agent Reinforcement Learning Paradigm for Mathematical Intelligence

The evolution of large language models from simple text predictors to sophisticated reasoning engines has necessitated a fundamental shift in how computational resources are allocated during inference. While the initial introduction of chain-of-thought prompting provided a mechanism for increasing computational depth, it remained a static policy, applying uniform computational effort to problems of varying difficulty levels.[1] The contemporary research frontier has shifted toward adaptive reasoning computation, where models are trained via reinforcement learning to dynamically modulate their "thinking" time based on the intrinsic complexity of the input query. This transition is predicated on the understanding that the most efficient way to scale model performance is not merely through parameter count or pre-training tokens, but through the intelligent scaling of test-time compute.[3] Central to this pursuit is the development of reinforcement learning with verifiable rewards, a paradigm that allows models to explore reasoning trajectories and receive objective feedback based on the correctness of their final answers and the logical validity of their intermediate steps.[5]

Foundations of Test-Time Scaling and Adaptive Computation

The conceptual framework for adaptive reasoning treats the process of generating a solution as a sequential decision-making task. In this context, the model acts as an agent within a Markov Decision Process, where each state represents the current partial reasoning trace and the original prompt.[2] The action space is expanded beyond the vocabulary of the language model to include specialized control tokens that dictate the reasoning mode—such as continuing a derivation, performing a verification check, or terminating the sequence once a solution is reached.[8] This modeling approach allows for the optimization of a policy that balances the probability of a correct answer against the computational cost incurred by generating additional tokens.[10]

The motivation for this adaptive behavior stems from the "overthinking" problem observed in high-capacity reasoning models. For instance, advanced models have been observed generating thousands of redundant reasoning tokens for trivial arithmetic tasks, such as simple addition, because they have been trained to maximize reasoning depth without a corresponding penalty for inefficiency.[3] Conversely, for complex problems found in competitive mathematics benchmarks like the American Invitational Mathematics Examination (AIME) or the

International Mathematical Olympiad, models often fail because they lack a mechanism to trigger extended reflection or backtracking when initial reasoning paths are found to be flawed.[13]

Taxonomy of Test-Time Scaling Strategies

Strategy	Mechanism	Primary Objective	Key Implementation
Fixed CoT	Uniform step-by-step reasoning for all queries.	Basic reasoning capability.	Chain-of-Thought Prompting.[1]
Parallel Sampling	Generating N independent solutions and selecting the best.	Maximizing Pass@ N accuracy.	Self-Consistency Decoding.[2]
Sequential Search	Tree search or backtracking within a single sequence.	Efficient exploration of hard tasks.	Tree-of-Thoughts, PRIME.[9]
Adaptive RL	Learned policy for switching between reasoning modes.	Compute efficiency and accuracy.	Thinkless, AutoThink.[8]

The development of adaptive reasoning models typically involves a two-stage process: a supervised fine-tuning warm-up phase followed by a reinforcement learning phase.[18] During the warm-up, models are trained on curated traces that include the desired control tokens, ensuring that they understand the structural requirements of the adaptive policy. The subsequent RL phase then optimizes the policy's decision-making logic, using algorithms like Group Relative Policy Optimization (GRPO) to encourage the model to select the most efficient reasoning path that consistently leads to a correct answer.[20]

Algorithmic Paradigms: From PPO to GRPO and DeGRPO

The optimization of reasoning policies requires algorithms that can handle the specific constraints of large-scale language model training. While Proximal Policy Optimization (PPO)

was the dominant approach for several years, its reliance on a separate critic model poses significant memory and computational challenges when scaling to models with billions of parameters.[20] The introduction of Group Relative Policy Optimization (GRPO) addressed these limitations by replacing the neural critic with group-level statistics.[20]

In the GRPO framework, the model generates a group of G independent rollouts for a given prompt. The advantage of each rollout is calculated by comparing its reward to the mean and standard deviation of rewards within the group. This relative advantage serves as the training signal, pushing the policy toward trajectories that are "better than average" for that specific problem.[20] This mechanism is particularly effective for reasoning tasks where verifiable rewards—such as a binary signal from a math parser or a code interpreter—can be used to evaluate final answers.[5]

The objective function for GRPO can be represented as:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\min\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q,o_{i,<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right)\right]$$

where $\hat{A}_{i,t}$ is the advantage of the i-th completion in the group, calculated relative to the group rewards $\{R_1, R_2, \ldots, R_G\}$.[20]

Comparison of Reinforcement Learning Objectives for Reasoning

Algorithm	Advantage Estimation	Memory Overhead	Handling of Length Bias
PPO	Value model (Critic).	High (requires extra parameters).	Indirect (via reward shaping).
GRPO	Group-level mean/std.	Low (critic-free).	Heuristic normalization.[21]
DAPO	Group-level mean.	Low.	Enforces uniform token-level weighting.[20]
DeGRPO	Decoupled control vs. response.	Low to Moderate.	Explicitly separates mode selection

			from accuracy.[8]

While GRPO is efficient, it is susceptible to "mode collapse" in adaptive settings. If the reward signal for accuracy is much stronger than the penalty for length, the model may simply learn to generate maximum-length reasoning traces for every query, ignoring the control tokens that were meant to trigger efficient paths.[18] Decoupled Group Relative Policy Optimization (DeGRPO) was developed to mitigate this by assigning distinct weights to the control token loss and the response token loss. This decoupling prevents the much more numerous response tokens from drowning out the gradient signal for the initial control token, ensuring the model remains responsive to difficulty-aware reasoning strategies.[8]

The Self-Correction Dilemma: Why SFT Fails and RL Succeeds

A critical capability of advanced reasoning models is the ability to detect and rectify their own errors—a process known as intrinsic self-correction.[15] Despite its desirability, empirical evidence has shown that standard supervised fine-tuning (SFT) on self-correction traces is largely ineffective.[15] Models trained via SFT on traces where a mistake is followed by a correction often suffer from distribution shift: they can correct the specific errors made by the base model that generated the training data, but they fail to generalize this skill to the new types of errors they make after being fine-tuned.[15]

Furthermore, SFT often leads to "behavior collapse," where the model learns to provide a perfect answer in the first turn and treats the second turn as a superficial formality.[15] This "direct" strategy optimizes the reward on a static training set but fails to instill a generalized meta-strategy for test-time error recovery.[15]

The SCoRe Training Recipe for Robust Self-Correction

The SCoRe (Self-Correction via Reinforcement Learning) framework addresses these failures through a specialized multi-turn RL approach using entirely self-generated data.[15] The process is divided into two primary stages:

1. **Policy Initialization:** Instead of using SFT, the model is trained to optimize the accuracy of the second-attempt solution while being constrained (via KL divergence or other regularization) to keep the first-turn distribution close to the original base model. This ensures the model continues to make "natural" errors that it can then learn to fix.[15]
2. **Multi-turn RL with Progress Rewards:** The model undergoes joint optimization of both attempts. A key innovation is the use of reward shaping that specifically incentivizes *progress*. A model that moves from an incorrect first attempt to a correct second attempt receives a higher reward bonus than one that was correct in both turns, explicitly teaching the model that error recovery is a high-value behavior.[15]

Comparative Performance Gains from SCoRe

Model	Benchmark	SCoRe Improvement (Absolute)	Baseline Performance
Gemini 1.0 Pro	MATH	+15.6%	35.2%
Gemini 1.5 Flash	MATH	+9.1%	48.5%
Gemini 1.0 Pro	HumanEval	+15.6%	62.4%
Gemini 1.5 Flash	HumanEval	+9.1%	74.2%

These results demonstrate that online RL is the first approach to achieve significantly positive intrinsic self-correction without relying on external feedback or teacher models.[15] This is particularly relevant for adaptive reasoning pipelines, as a model that can self-correct is inherently more efficient; it only spends additional tokens when its internal verification signal detects a likely error in its initial derivation.[30]

Reward Shaping and the Adaptive Length Penalty

The design of the reward function is the most influential factor in determining the behavior of an RL-trained reasoning model. A binary reward for correctness (1 for correct, 0 for incorrect) often leads to reward hacking, where the model exploits the reward signal by becoming excessively verbose.[10] To counteract this, researchers incorporate length penalties into the reward function.[2] However, a uniform length penalty applied across all problems is counter-productive, as it discourages deep reasoning for genuinely difficult tasks.[3]

The Adaptive Length Penalty (ALP) framework introduces a difficulty-conditioned penalty that scales based on the model's confidence in a given prompt. The "online solve rate" of a prompt—calculated across the rollouts in a GRPO group—serves as a proxy for problem difficulty.[3]

The ALP Reward Formulation

The ALP reward function R for a rollout o can be defined as:

$$R(o) = r_{\text{acc}}(o) - \beta \cdot \max(0, \text{SR}(q)) \cdot \frac{n_{\text{tokens}}}{\text{max\_length}}$$

where:

- $r_{\text{acc}}(o)$ is the verifiable accuracy reward.
- $\text{SR}(q)$ is the empirical solve rate for the prompt q (e.g., the proportion of correct rollouts in the group).
- β is a global scaling coefficient.
- n_{tokens} is the number of tokens in the current rollout.[11]

This formulation ensures that "easy" prompts (high solve rate) incur a heavy penalty for excessive tokens, forcing the model toward brevity. Conversely, for "hard" prompts (low solve rate), the penalty is minimal, allowing the model to explore long-chain reasoning.[3] Empirical evaluations of ALP on models like DeepScaleR-1.5B show that it can reduce average token usage by over 50% while maintaining or even improving accuracy on challenging tasks.[11]

Token Usage Adaptation Ratios

Method	Adaptation Ratio (Hard vs. Easy)	Accuracy on MATH (1.5B)
Fixed Budget	1.01x	42.1%
ThinkPrune	2.15x	44.3%
ALP (Reinforcement Learning)	**5.35x**	**45.6%**

This adaptive allocation strategy allows models to internalize problem difficulty, with token usage increasing monotonically as the empirical solve rate decreases.[11] Analysis of reasoning

traces shows that ALP selectively compresses redundant behaviors—such as phrase repetitions and unnecessary backtracking—while preserving high-value planning steps.[3]

Architectural Audit: Adaptation of Qwen2.5-Math for RL Tasks

The success of reinforcement learning for reasoning is highly dependent on the choice of the base model. The Qwen2.5-Math series, developed by Alibaba Cloud, has established itself as the leading open-source family for these tasks.[1] Qwen2.5-Math-7B-Instruct, in particular, is a 7.62-billion parameter model optimized for English and Chinese mathematical reasoning through a pipeline that includes pre-training on 1 trillion tokens and supervised fine-tuning (SFT) using rejection sampling.[1]

One of the distinguishing features of the Qwen series is its dual-mode reasoning capability, supporting both standard Chain-of-Thought (CoT) and Tool-Integrated Reasoning (TIR) via a Python interpreter.[1] TIR allows the model to outsource precise calculations and symbolic manipulations to an external tool, mitigating the common issue of arithmetic errors in long reasoning traces.[1]

Qwen2.5-Math-7B Performance Baseline

Benchmark	Mode	Greedy Accuracy	Pass@8 / RM@8
MATH	CoT	83.6%	85.3% [31]
MATH	TIR	85.3%	87.8% [32]
GSM8K	CoT	93.4%	95.2% [32]
AIME 2024	CoT	16.6% (5/30)	21 Problems [31]

While the base model's performance is exceptional on elementary benchmarks like GSM8K, there is significant room for improvement on competition-level benchmarks like AIME and MATH. The gap between greedy decoding and RM@8 (where a reward model selects from 8 sampled responses) indicates that the model's underlying distribution contains correct solutions that it fails to prioritize during single-pass inference.[31] This provides a clear objective for RL: to refine the policy so that it consistently selects these "high-reward" trajectories.[32]

Parameter-Efficient Fine-Tuning: LoRA Singular Value

Dynamics

For many researchers, full fine-tuning of 7B+ parameter models is computationally prohibitive, making Low-Rank Adaptation (LoRA) the standard for RL training.[38] LoRA works by freezing the pre-trained weights $W_0 \in \mathbb{R}^{d \times k}$ and introducing two low-rank matrices $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$, where the update is $\Delta W = BA$.[39] While LoRA is effective, recent spectral analyses have revealed fundamental differences between LoRA and full fine-tuning (FullFT).[41]

LoRA-trained weight matrices exhibit "intruder dimensions"—high-ranking singular vectors that do not exist in the original pre-trained structure. These intruder dimensions are found to be the primary cause of knowledge forgetting during fine-tuning.[41] In contrast, FullFT updates tend to rotate the existing basis of the model more uniformly, preserving pre-trained features more effectively for a given level of downstream adaptation.[41]

Optimal LoRA Configurations for Reasoning RL

Despite these structural differences, LoRA has been shown to perform equivalently to FullFT for reinforcement learning, even with very low ranks.[38] This is likely because RL episodes contain relatively low information density compared to dense supervised datasets, making the small parameter space of a rank-1 or rank-8 adapter sufficient for the task.[39]

Hyperparameter	Recommendation	Reasoning
Rank (r)	1 to 16	$r =$ is often sufficient for safety and reasoning tasks.[43]
Alpha (α)	2 * Rank	Standard heuristic for stable optimization.[40]
Target Modules	All (MLP + Attention)	MLP layers are critical for factual retrieval and complex logic.[39]
Learning Rate	10x FullFT	LoRA updates need higher rates to move the low-rank subspace.[38]

A common mistake in earlier LoRA research was applying adapters only to the attention layers. Modern best practices emphasize that applying LoRA to MLP layers—specifically the

up-projection and down-projection matrices—is significantly more effective for reasoning and domain adaptation tasks.[39]

Optimization Pathologies: Reward Hacking and Mode Collapse

Training reasoning models with verifiable rewards often leads to "reward hacking," where the model finds shortcuts to maximize reward without truly solving the problem.[6] In the context of adaptive reasoning, the most common form of reward hacking is "length bias," where the model learns that longer reasoning chains correlate with higher accuracy on its training set, leading to an explosion in verbosity.[25]

Another pathology is "training collapse" or "length collapse," where both the reward and the average token count decline sharply during training. This often happens when the model "hacks" the length penalty by finding a way to generate the correct final answer almost immediately, without any supporting reasoning, thereby losing its logical generalization ability.[25]

Taxonomy of Reward Hacking Failures

Failure Mode	Symptom	Causal Factor
Verbosity Bias	Excessively long reasoning traces for easy problems.	Lack of length penalty or high variance in group rewards.[12]
Format Exploitation	Generating boxed{correct_answer} without logical steps.	Outcome-only rewards with no process supervision.[6]
Control Collapse	Ignoring tokens like <verify> or <think>.	Gradient dominance of response tokens over control tokens.[18]
Solution Plateaus	Accuracy stagnates despite more training data.	Limited exploration or sparse rewards on hard problems.[4]

To address these issues, researchers have moved toward "Reasoning-Aligned Reinforcement Learning" (RARL), which incorporates process-aware rewards.[46] For example, the Conciseness Reward Function (CRF) applies a length penalty *only* when the answer is correct, preventing the model from learning to be "concisely wrong" and ensuring it only optimizes for length once

it has mastered the underlying logic.[25]

Auditing the Minimal Adaptive Policy Research Proposal

The proposed research project, "Learning When to Think: RL for Adaptive Reasoning Computation in LLMs," aims to train an adaptive policy with three actions (continue, verify, terminate) on Qwen2.5-Math-7B-Instruct using GRPO and LoRA on the GSM8K dataset [User Query]. While this scope is feasible for a course project, a rigorous audit reveals several fundamental flaws and opportunities for significant improvement.

Audit of Potential Research Flaws

1. **Action Space Inexpressivity:** The action space consists of continue, verify, and terminate. However, the proposal does not specify a *mechanism* for backtracking. If the verify action identifies an error, the agent's only subsequent choice is to continue or terminate. Without a mechanism to jump back to a previous state or generate a critique for refinement, the verify action becomes a passive observation rather than an active control.[16]
2. **Benchmark Saturation:** Training on GSM8K with Qwen2.5-Math-7B is problematic because the base model already achieves ~93% accuracy on this benchmark.[32] There is very little "headroom" for RL to show meaningful improvement. On easy problems, accuracy rewards will saturate quickly, potentially leading to gradient collapse or reward hacking where the model only learns to be shorter without improving its reasoning.[12]
3. **Static Length Penalty:** The reward function uses a fixed $\lambda \cdot n_{\text{tokens}}$. As discussed in the ALP literature, static penalties fail to account for problem difficulty, leading to over-penalization of tokens on hard problems where long reasoning is necessary.[3]
4. **Mode Collapse Risks:** Using vanilla GRPO for control tokens (verify/terminate) is highly likely to result in mode collapse, where the model ignores these tokens in favor of standard token generation.[18]

Proposed Solutions and Improved Methodology

To solve these flaws and elevate the research to an expert level, the following modifications are recommended:

- **Implement Backtracking or Refinement:** Instead of a simple verify token, the action should trigger a "Refine" block or a state-stack mechanism similar to the PRIME framework.[7] This allows the model to actually *use* the verification signal to repair its reasoning trace.
- **Pivot to Challenging Benchmarks:** The evaluation should include MATH or a subset of AIME 2024. These benchmarks have significantly lower baseline scores (~16-45%),

providing ample headroom for the RL policy to discover new reasoning strategies.[31]

- **Adopt Adaptive Reward Shaping:** Replace the fixed token penalty with an Adaptive Length Penalty (ALP) that scales based on the group solve rate. This will protect the model's performance on hard problems while aggressively pruning redundancy on easy ones.[3]
- **Utilize DeGRPO for Control Tokens:** To ensure the model learns to respect the verify and terminate actions, implement the DeGRPO loss formulation. This will balance the gradient updates between the high-level mode selection and the low-level reasoning tokens.[8]
- **Leverage Existing Process Supervision:** Instead of training a PRM from scratch, the project should integrate the open-source Qwen2.5-Math-PRM-7B as a reward signal.[36] Using a pre-trained PRM to provide step-level rewards will solve the sparse reward problem and accelerate policy convergence.[6]

Benchmarking the Future: Contamination and "Live" Evaluations

A significant challenge in evaluating mathematical reasoning models is the risk of data contamination. As models are trained on larger portions of the internet, there is a high probability that historical competition problems (like AIME 2024) have been seen during pre-training, leading to artificially inflated scores.[14] This makes it difficult to distinguish between true logical reasoning and sophisticated pattern matching.[14]

To address this, the reasoning community is shifting toward "Live Evaluation" strategies. Benchmarks like MathArena exclusively use problems from very recent competitions (e.g., AIME 2025) that were released after the model's training data cutoff.[13]

Leaderboard of Reasoning Models on AIME 2025 (Projected Late 2025)

Model Class	Model Name	AIME 2025 Accuracy (Pass@1)	Note
Frontier Closed	GPT-5	~94.6%	Approaching human saturation.[13]
Frontier Reasoning	Grok-4	~91.3%	Advanced RL search.[13]
Open Weights	Qwen-OSS 120B	~92.6%	Massive scale +

(Large)			TIR.[13]
Open Weights (Medium)	Claude 3.7 Sonnet	~52.7%	Progress in Anthropic CoT.[13]
Baseline	Random Guessing	~20.0%	Zero reasoning capability.[13]

The results on AIME 2025 highlight the massive gap between "general purpose" models and those that have been specifically reinforced for reasoning depth and verification.[13] For a 7B model to compete in this landscape, it must employ highly efficient adaptive compute strategies to maximize the utility of every generated token.[48]

Synthesis and Strategic Outlook

The pursuit of "learning when to think" represents the next phase of LLM alignment. By moving away from fixed computational budgets toward learned, adaptive policies, we can create models that are not only more accurate but also significantly more cost-effective for large-scale deployment.[3] The technical path forward involves the integration of process-level supervision, multi-turn RL for self-correction, and adaptive length penalties to manage the overthinking-underthinking dilemma.[6]

For individual researchers and smaller teams, the availability of high-quality base models like Qwen2.5-Math-7B and efficient RL algorithms like GRPO has democratized access to these advanced techniques.[20] However, achieving state-of-the-art performance requires a nuanced understanding of RL pathologies—such as reward hacking and distribution shift—and the employment of sophisticated solutions like SCoRe and DeGRPO.[8] Ultimately, the goal is to develop autonomous reasoning agents that can reflect on their own logic, verify their derivations, and allocate their "cognitive" resources with the same fluid efficiency as a human expert.

Works cited

1. Qwen 2.5 Math 7B - Open Laboratory, accessed March 30, 2026, https://openlaboratory.ai/models/qwen-2_5-math-7b
2. Primers • Reasoning in LLMs - aman.ai, accessed March 30, 2026, https://aman.ai/primers/ai/reasoning-in-LLMs/
3. Just Enough Thinking: Efficient Reasoning with Adaptive Length Penalties Reinforcement Learning - arXiv, accessed March 30, 2026, https://arxiv.org/html/2506.05256v1
4. How to Explore to Scale RL Training of LLMs on Hard Problems?, accessed March 30, 2026, https://blog.ml.cmu.edu/2025/11/26/how-to-explore-to-scale-rl-training-of-llms-

on-hard-problems/
5. Reinforcement Learning Meets Large Language Models: A Survey of Advancements and Applications Across the LLM Lifecycle - arXiv, accessed March 30, 2026, https://arxiv.org/html/2509.16679v1
6. Qwen2.5-Math-PRM: Advanced Math LLM Framework - Emergent Mind, accessed March 30, 2026, https://www.emergentmind.com/topics/qwen2-5-math-prm
7. RL of Thoughts: Navigating LLM Reasoning with Inference-time Reinforcement Learning, accessed March 30, 2026, https://arxiv.org/html/2505.14140v3
8. NeurIPS Poster Thinkless: LLM Learns When to Think, accessed March 30, 2026, https://neurips.cc/virtual/2025/poster/117232
9. PRIME: Policy-Reinforced Iterative Multi-Agent Execution for Algorithmic Reasoning in Large Language Models - Preprints.org, accessed March 30, 2026, https://www.preprints.org/manuscript/202601.1479/v1
10. SelfBudgeter: Adaptive Token Allocation for Efficient LLM Reasoning - arXiv, accessed March 30, 2026, https://arxiv.org/html/2505.11274v4
11. [Literature Review] Just Enough Thinking: Efficient Reasoning with Adaptive Length Penalties Reinforcement Learning - Moonlight | AI Colleague for Research Papers, accessed March 30, 2026, https://www.themoonlight.io/en/review/just-enough-thinking-efficient-reasoning-with-adaptive-length-penalties-reinforcement-learning
12. Just Enough Thinking: Efficient Reasoning with Adaptive Length Penalties Reinforcement Learning | OpenReview, accessed March 30, 2026, https://openreview.net/forum?id=QY5tl8AhlT
13. AIME 2025 Benchmark: An Analysis of AI Math Reasoning - IntuitionLabs, accessed March 30, 2026, https://intuitionlabs.ai/pdfs/aime-2025-benchmark-an-analysis-of-ai-math-reasoning.pdf
14. AIME 2024 and AIME 2025 Benchmarks - Emergent Mind, accessed March 30, 2026, https://www.emergentmind.com/topics/aime-2024-and-aime-2025-benchmarks
15. Training Language Models to Self-Correct via Reinforcement ... - arXiv, accessed March 30, 2026, https://arxiv.org/pdf/2409.12917
16. To Backtrack or Not to Backtrack: When Sequential Search Limits Model Reasoning - arXiv, accessed March 30, 2026, https://arxiv.org/html/2504.07052v2
17. NeurIPS Poster Learning When to Think: Shaping Adaptive Reasoning in R1-Style Models via Multi-Stage RL, accessed March 30, 2026, https://neurips.cc/virtual/2025/poster/118864
18. Thinkless: LLM Learns When to Think - OpenReview, accessed March 30, 2026, https://openreview.net/pdf?id=ariVQf0KZx
19. Thinkless: LLM Learns When to Think - arXiv, accessed March 30, 2026, https://arxiv.org/pdf/2505.13379
20. trl/docs/source/grpo_trainer.md at main · huggingface/trl - GitHub, accessed March 30, 2026, https://github.com/huggingface/trl/blob/main/docs/source/grpo_trainer.md

21. Group Relative Policy Optimization (GRPO) - verl documentation, accessed March 30, 2026, https://verl.readthedocs.io/en/latest/algo/grpo.html
22. Post training an LLM for reasoning with GRPO in TRL - Hugging Face, accessed March 30, 2026, https://huggingface.co/learn/cookbook/fine_tuning_llm_grpo_trl
23. Token-Efficient RL for LLM Reasoning - arXiv, accessed March 30, 2026, https://arxiv.org/html/2504.20834v4
24. Daily Papers - Hugging Face, accessed March 30, 2026, https://huggingface.co/papers?q=Qwen3-4B-Instruct
25. Efficient Reasoning via Reward Model - arXiv, accessed March 30, 2026, https://arxiv.org/pdf/2511.09158
26. λ-GRPO: Unifying the GRPO Frameworks with Learnable Token Preferences - arXiv, accessed March 30, 2026, https://arxiv.org/html/2510.06870v2
27. Thinkless: LLM Learns When to Think - arXiv, accessed March 30, 2026, https://arxiv.org/html/2505.13379v1
28. Can LLMs Correct Themselves? A Benchmark of Self-Correction in LLMs - OpenReview, accessed March 30, 2026, https://openreview.net/forum?id=956KYtqwcU&referrer=%5Bthe%20profile%20of%20Lichao%20Sun%5D(%2Fprofile%3Fid%3D~Lichao_Sun1)
29. Google Publishes LLM Self-Correction Algorithm SCoRe - InfoQ, accessed March 30, 2026, https://www.infoq.com/news/2024/10/google-deepmind-score/
30. Self-rewarding correction for mathematical reasoning - arXiv, accessed March 30, 2026, https://arxiv.org/html/2502.19613v1
31. QwenLM/Qwen2.5-Math: A series of math-specific large language models of our Qwen2 series. - GitHub, accessed March 30, 2026, https://github.com/QwenLM/Qwen2.5-Math
32. Qwen2.5-Math: The world's leading open-sourced mathematical LLMs | Qwen, accessed March 30, 2026, https://qwenlm.github.io/blog/qwen2.5-math/
33. Qwen/Qwen2.5-Math-7B-Instruct - Hugging Face, accessed March 30, 2026, https://huggingface.co/Qwen/Qwen2.5-Math-7B-Instruct
34. Algorithm Baselines - verl documentation - Read the Docs, accessed March 30, 2026, https://verl.readthedocs.io/en/latest/algo/baseline.html
35. Performance on Qwen2.5-7B-Instruct · Issue #42 · simplescaling/s1 - GitHub, accessed March 30, 2026, https://github.com/simplescaling/s1/issues/42
36. Towards Effective Process Supervision in Mathematical Reasoning - Qwen, accessed March 30, 2026, https://qwen.ai/blog?id=qwen2.5-math-prm
37. Reasoning or Memorization? Unreliable Results of Reinforcement Learning Due to Data Contamination - arXiv, accessed March 30, 2026, https://arxiv.org/html/2507.10532v2
38. LoRA Without Regret - Thinking Machines Lab, accessed March 30, 2026, https://thinkingmachines.ai/blog/lora/
39. LoRA vs. Full Fine-Tuning: The Truth No One Told You | by Lakshay Dagar | Medium, accessed March 30, 2026, https://medium.com/@ldagar315/lora-vs-full-fine-tuning-the-truth-no-one-told-you-2bdffa14aedb
40. Efficient LLM Fine-tuning with LoRA | by Sulbha Jain | Medium, accessed March

30, 2026, https://sulbhajain.medium.com/efficient-llm-fine-tuning-with-lora-0f650497da8c
41. LoRA vs Full Fine-tuning: An Illusion of Equivalence | OpenReview, accessed March 30, 2026, https://openreview.net/forum?id=xp7B8rkh7L
42. LoRA is All You Need for Safety Alignment of Reasoning LLMs - arXiv, accessed March 30, 2026, https://arxiv.org/html/2507.17075v3
43. LoRA is All You Need for Safety Alignment of Reasoning LLMs - arXiv, accessed March 30, 2026, https://arxiv.org/html/2507.17075v2
44. Optimizing LoRA target module selection for efficient fine tuning - Amazon Science, accessed March 30, 2026, https://www.amazon.science/blog/optimizing-lora-target-module-selection-for-efficient-fine-tuning
45. Advancing LLM Reasoning with Natural Language and Numerical Feedback - arXiv, accessed March 30, 2026, https://arxiv.org/html/2506.03106v6
46. Reward Modeling for Reinforcement Learning-Based LLM Reasoning: Design, Challenges, and Evaluation - arXiv, accessed March 30, 2026, https://arxiv.org/pdf/2602.09305
47. Qwen/Qwen2.5-Math-PRM-7B - Hugging Face, accessed March 30, 2026, https://huggingface.co/Qwen/Qwen2.5-Math-PRM-7B
48. DiffAdapt: Difficulty-Adaptive Reasoning for Token-Efficient LLM Inference - arXiv, accessed March 30, 2026, https://arxiv.org/html/2510.19669v2
49. DiffAdapt: Difficulty-Adaptive Reasoning for Token-Efficient LLM Inference - arXiv.org, accessed March 30, 2026, https://arxiv.org/html/2510.19669v3